Master Specification

1. Product goal

FinWallet is a .NET 8 multi-currency digital-wallet side project intended to demonstrate real financial-backend concerns: transaction consistency, double-entry accounting, idempotency, concurrency, external integrations, fraud, authentication/session security, cutoff, campaigns, notifications, reconciliation, security and observability.

2. Current scope

Customer/Auth

- Registration with TR/AZ country/phone policy.
- Redis-backed OTP verification.
- PBKDF2 password hashing.
- JWT access token plus durable server session.
- Opaque refresh-token rotation and reuse detection.

Wallet/Account

- TRY/USD/EUR wallet create/list.
- External BankAccount opening through FakeBank.
- Wallet and BankAccount are separate domain concepts.
- No FX conversion in v1.

Financial Core

- Double-entry Ledger.
- Durable FinancialTransaction.
- MSSQL-backed durable idempotency.
- Atomic wallet-to-wallet transfer.
- Internal plus external fraud decision.

Platform

- YARP Gateway is the entry point for normal public/client and FinWallet-to-provider HTTP traffic.
- Gateway applies JWT, internal-service authorization, rate limiting, health checks and load balancing.
- Every Web API has Swagger; production defaults to disabled.

3. Not yet complete

- Public BankDeposit and BankWithdrawal.
- Merchant purchase / campaign accounting.
- Public Refund/Reversal flow.
- Durable FraudEvents/manual review.
- Transactional Outbox/Inbox.
- Transaction history/read model.
- ReconciliationRuns/ReconciliationIssues.

- Centralized masked structured logging, OpenTelemetry and alerting.
- Real MSSQL/Redis/YARP integration-concurrency test suite.
- Public logout/session-revoke endpoint.

4. Technology rules

- .NET 8 / C# 12.
- ASP.NET Core controller-based Web API.
- MSSQL as durable source of truth.
- Redis for transient state only.
- JWT auth; no ASP.NET Core Identity.
- YARP reverse proxy/gateway.
- Built-in DI.
- Paid/freemium NuGet packages are forbidden.
- Package versions are centrally managed in Directory.Packages.props.

5. Architecture

The main application is a modular monolith:

```
FinWallet.Api
FinWallet.Application
FinWallet.Domain
FinWallet.Infrastructure
FinWallet.Shared.Contracts
FinWallet.Shared.Web
FinWallet.Gateway
```

External simulators are separate Web API processes:

```
FakeBank.Api
FakeFraud.Api
FakeCutoff.Api
FakeCampaign.Api
FakeCommunication.Api
```

6. Financial-correctness invariants

1. No money movement may bypass the Ledger.
2. Every posted journal must satisfy total Debit = total Credit.
3. MSSQL is the final financial-consistency authority.
4. Redis loss must not create duplicate money, negative balance or ledger corruption.
5. External HTTP never runs inside an open financial SQL transaction.
6. Duplicate commands and retries must be safe.
7. Same idempotency key + different payload is a conflict.
8. Completed financial history is not mutated/deleted; corrections use reversal/compensation.
9. Currency is part of every Money value and validated before commit.
10. Reconciliation never silently repairs a mismatch with a balance UPDATE.

7. Gateway security rule

```
Client -> Gateway -> FinWallet.Api
FinWallet.Api -> Gateway /providers/* -> Fake providers
```

- Gateway requires JWT for protected public /api/* routes.
- FinWallet.Api repeats JWT and ownership validation.
- FinWallet.Api calls provider routes using InternalServiceKey.
- Gateway writes a separate DownstreamServiceKey on destination requests.
- Backend/provider business endpoints reject direct calls without the downstream key.

8. Financial-operation model

Wallet transfer order:

```
completed durable replay
-> durable session/risk signals
-> internal fraud
-> external fraud
-> final Allow
-> atomic MSSQL posting
```

Atomic posting commits idempotency + balances + FinancialTransaction + LedgerJournal + LedgerEntries in one transaction.

9. Security rules

- Never log password/OTP/JWT/refresh token/service keys/connection secrets.
- Use parameterized SQL.
- Authorization/ownership does not trust client-supplied identity or risk flags.
- JWT signing algorithm is a code-level invariant.
- PBKDF2 scheme changes require versioned migration.
- Rate/body/header/connection/time limits exist at Gateway and backend layers.
- Volumetric DDoS additionally requires ingress/WAF/cloud DDoS controls.

10. Definition of Done

A feature is complete only when:

- the solution builds successfully;
- relevant tests pass;
- financial/concurrency/idempotency invariants remain valid;
- external failure behavior is defined;
- security/logging impact is reviewed;
- package inventory is current;
- TR/EN XML documentation is current;
- all affected Markdown documents are TR+EN and consistent with code.

11. Explicitly out of scope for v1

- real bank/fraud/SMS/email providers;
- credit-card acquiring/payment gateway;
- full core banking;
- loans/credit scoring;
- stock/crypto trading;
- FX conversion;

- Event Sourcing;
- mandatory Kafka/RabbitMQ;
- full microservice decomposition.